

스노우 보드샵

- 온라인 주문 홈페이지

이아름

프로젝트 개요

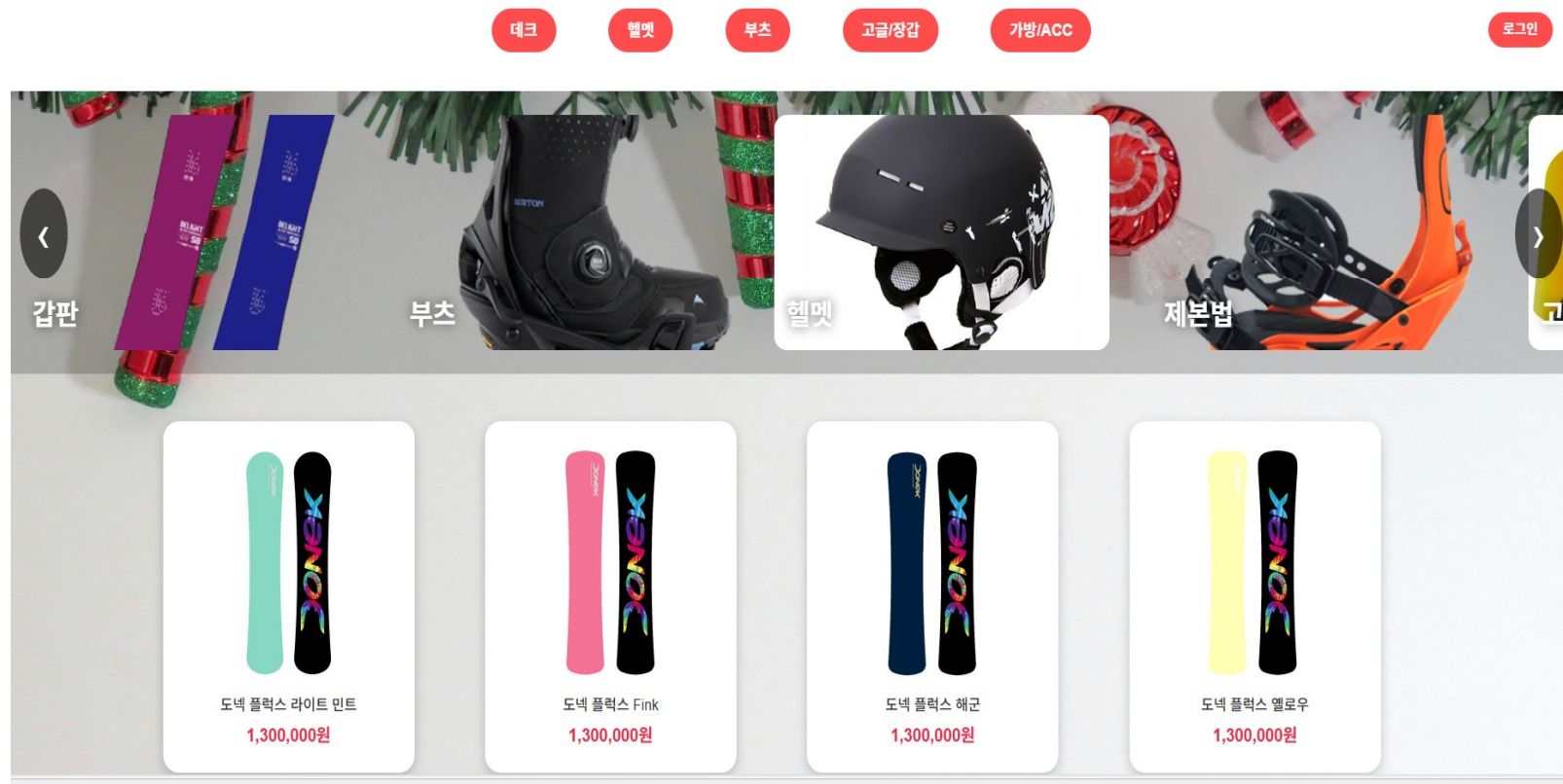
- React를 활용한 온라인 쇼핑몰 구현
- 회원가입 / 로그인 / 장바구니 / 구매 흐름 구현
- 실제 쇼핑몰 UI/UX 흐름을 학습하기 위한 프로젝트

- 사용 기술

- React
- JavaScript (ES6)
- CSS
- LocalStorage
- GitHub Pages

| 홈 화면 구성

- 메인 상품 슬라이더
- 카테고리 버튼
- 로그인 / 로그아웃 버튼



| 로그인 / 회원가입 시스템

- localStorage 기반 회원가입 구현
- 회원가입안 사용자만 로그인 가능
- 로그인 상태 유지 및 로그아웃 처리

A login form titled '로그인' (Login) with a close button (X) in the top right corner. It contains two input fields: '아이디' (ID) and '비밀번호' (Password). Below the fields is a red button labeled '로그인' (Login). At the bottom left, there is a link labeled '회원가입' (Sign Up) circled in red. An arrow points from this link to the registration form on the right.

A registration form titled '회원가입' (Sign Up) with a close button (X) in the top right corner. It contains three input fields: '아이디' (ID), '비밀번호' (Password), and '비밀번호 확인' (Confirm Password). Below the fields is a red button labeled '회원가입' (Sign Up). At the bottom left, there is a link labeled '로그인으로' (Login).

A message box with a title 'localhost:3000의 메시지' (Message from localhost:3000). The text inside says '아이디 또는 비밀번호가 틀렸습니다.' (ID or password is incorrect). There is a blue button labeled '확인' (Confirm) in the bottom right corner.

A login form titled '로그인' (Login) with a close button (X) in the top right corner. It contains two input fields: '아이디' (ID) and '비밀번호' (Password). Below the fields is a red button labeled '로그인' (Login). At the bottom left, there is a link labeled '회원가입' (Sign Up).

| 장바구니 기능

- 상품 추가 및 수량 증가 처리
- 로그인 상태 유지와 연동 가능



▼ 수량 증가 기능

```
const addToCart = (product) => {
  setCart((prev) => {
    const exists = prev.find((item) => item.id === product.id);

    if (exists) {
      return prev.map((item) =>
        item.id === product.id
          ? { ...item, quantity: item.quantity + 1 }
          : item
      );
    }

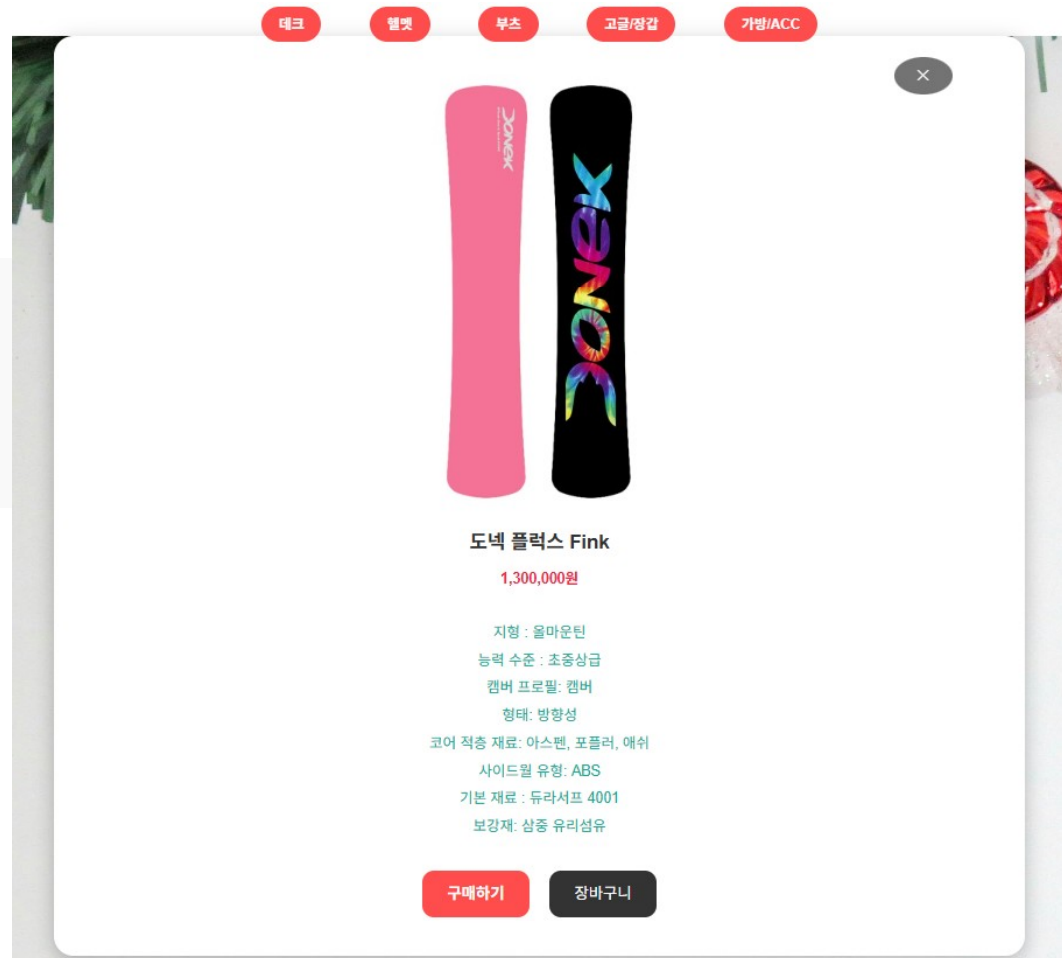
    return [...prev, { ...product, quantity: 1 }];
  });
};
```

| 상품 리스트 & 상세 페이지

- 상품 클릭 시 상세 페이지 이동
- 상품 상세 페이지
- 상품 이미지
- 상품명 / 가격
- 장바구니 담기 버튼

장바구니 구매 처리

```
<Cart  
  cart={cart}  
  setCart={setCart}  
  onPurchase={() => setIsPurchased(true)}  
>
```



| 구매 흐름 (에러 없이 화면만 구현)

- 실제 결제는 구현하지 않음
- 구매 버튼 클릭 시 구매 완료 화면 출력
-  구매 완료
- 구매 완료 메시지 표시
- 홈으로 돌아가기 버튼 제공

★ 버튼클릭(CART.JS)

- 구매하기 버튼 클릭 시 onPurchase 함수 실행
- App 컴포넌트에서 전달받은 함수로 구매 상태 변

```
<button onClick={onPurchase}>  
  구매하기  
</button>
```

★ 구매 상태 관리 (App.js)

- isPurchased 상태값으로 구매 여부 관리
- 초기값은 false
- 구매 버튼 클릭 시 true로 변경

```
const [isPurchased, setIsPurchased] = useState(false);
```

★ 구매 완료 처리 함수

- Cart 컴포넌트에서 전달받아 사용
- 구매 버튼 클릭 시 호출됨

```
const handlePurchase = () => {  
  setIsPurchased(true);  
};
```

★ 구매 완료 화면 조건 렌더링

- 구매 상태에 따라 화면 전환
- 페이지 이동 없이 SPA 방식으로 처리

```
{isPurchased ? (  
  <div className="purchase-complete">  
    <h2>🎉 구매가 완료되었습니다!</h2>  
    <p>주문해주셔서 감사합니다.</p>  
  </div>  
) : (  
  <Cart
```

★ 구매 후 장바구니 초기화

- 구매 완료 후 장바구니 비우기
- 다음 주문을 위한 UX 개선

```
const handlePurchase = () => {  
  setIsPurchased(true);  
  setCart([]);  
};
```


| 마무리

- React 핵심 개념 활용 (useState, props)
- 실제 쇼핑몰 흐름에 맞춘 구조 설계
- 추후 서버 연동 및 결제 기능 확장 가능